

Wazuh + SMS Alerting Integration

Real-Time Notifications via RT Communications API

Complete Step-by-Step Setup Guide

Author: Rakibul Islam Joy

Security Engineer | Unified IT

github.com/rakibuljoy

June 2026

Overview

This guide covers the complete setup of Wazuh SIEM integrated with the RT Communications SMS API to automatically send real-time SMS notifications when critical security events occur. Written from a real working environment — every command was tested and confirmed working.

What this setup does:

- Wazuh detects when an agent goes **OFFLINE** or comes back **ONLINE**
- Wazuh detects any security alert at **Level 12 or above** (critical severity)
- An SMS is sent immediately to the SOC engineer's phone via RT Communications API
- Cost-conscious design — only 2 event types trigger SMS to minimize API charges

Environment used:

- Wazuh Manager: Ubuntu Server 22.04 (hostname: wazuh-soc)
- Wazuh Agent: Ubuntu Desktop 22.04 (hostname: rakib-virtual-machine)
- Wazuh Version: 4.14.5
- SMS Gateway: RT Communications API (api.rtcom.xyz)

WARNING: The RT Communications API is a paid service. Each SMS costs money. This setup is intentionally designed to send SMS ONLY for agent connectivity changes and Level 12+ critical alerts — to avoid unnecessary charges.

Step 1 — Get RT Communications API Credentials

Log in to the RT Communications portal and collect the following four credentials. You will paste them directly into the Python script in Step 2.

- **acode** — Account Code (from portal dashboard)
- **api_key** — API Key (from portal dashboard)
- **senderid** — Sender ID (from portal dashboard)
- **contacts** — Your phone number in format: 8801XXXXXXXXXX

WARNING: Keep these credentials private. Never commit them to GitHub. Use placeholder values in any public repositories.

Step 2 — Create the Integration Script

Create the Python integration script on the **Wazuh Manager**.

```
# Run this on: MANAGER (wazuh-soc)
sudo nano /var/ossec/integrations/custom-sms-alert
```

WARNING: CRITICAL: The filename must NOT have a .py extension. Wazuh's integratord looks for the file by the exact value in ossec.conf. If you name it custom-sms-alert.py, Wazuh logs 'File not found inside integrations' and the integration silently never runs. Correct filename: custom-sms-alert (no extension).

Paste the complete script below:

```
#!/usr/bin/env python3
# Wazuh Integration – SMS Alert via RT Communications API
# File location: /var/ossec/integrations/custom-sms-alert
# Created by: Rakibul Islam Joy
# Purpose: Send SMS when agent goes offline/online or Level 12+ alert fires
#
# How it works:
# 1. Wazuh Manager detects a matching alert (agent ON/OFF or Level 12+)
# 2. integratord calls this script with the alert JSON file path as argv[1]
# 3. Script reads the JSON, determines which type of alert it is
# 4. Builds an appropriate SMS message and POSTs to RT Communications API
# 5. Logs the API response to sms_debug.log for debugging

import sys
import json
import requests
from datetime import datetime

try:
    # Step A: Read alert JSON file path sent by Wazuh
    alert_file_path = sys.argv[1]
    with open(alert_file_path, 'r') as f:
        alert_json = json.loads(f.readline())
```

```

# Step B: Extract alert fields from the JSON
rule_id      = alert_json.get('rule', {}).get('id', '0')
alert_level  = alert_json.get('rule', {}).get('level', '0')
alert_desc   = alert_json.get('rule', {}).get('description', 'No Description')
agent_name   = alert_json.get('agent', {}).get('name', 'Unknown')
timestamp    = datetime.now().strftime("%H:%M %d-%m-%Y")

# Step C: RT Communications API config – fill in your credentials below
payload = {
    "acode":          "YOUR_ACODE",          # From RT Communications portal
    "api_key":         "YOUR_API_KEY",        # From RT Communications portal
    "senderid":        "YOUR_SENDER_ID",      # From RT Communications portal
    "type":            "text",
    "msg":              "",                    # Built below based on rule type
    "contacts":         "8801XXXXXXXXX",      # Your phone e.g. 8801712345678
    "transactionType": "",
    "contentID":        ""
}

# Step D: Build SMS message based on which rule fired
if rule_id == '110600':
    # Agent went offline – triggered by our custom rule (child of rule 506)
    payload["msg"] = (
        f"[WAZUH ALERT]
"
        f"Agent OFFLINE!
"
        f"Agent: {agent_name}
"
        f"Time: {timestamp}"
    )
elif rule_id == '110601':
    # Agent came online – triggered by our custom rule (child of rule 503)
    payload["msg"] = (
        f"[WAZUH ALERT]
"
        f"Agent ONLINE
"
        f"Agent: {agent_name}
"
        f"Time: {timestamp}"
    )
else:
    # High severity alert Level 12+
    payload["msg"] = (
        f"[WAZUH HIGH ALERT]
"
        f"Level: {alert_level}

```

```

"
        f"Agent: {agent_name}"
"
        f"Rule: {rule_id}"
"
        f"Desc: {alert_desc[:60]}"
    )

# Step E: Send POST request to RT Communications API
headers = {'Content-Type': 'application/json'}
response = requests.post(
    "https://api.rtcom.xyz/onetomany",
    json=payload,
    headers=headers,
    timeout=10
)

# Step F: Log result for debugging
now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
with open("/var/ossec/logs/sms_debug.log", "a") as log:
    log.write(
        f"[{now}] RuleID:{rule_id} Level:{alert_level} "
        f"Agent:{agent_name} Status:{response.status_code} "
        f"Response:{response.text}\n"
    )

except Exception as e:
    now = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open("/var/ossec/logs/sms_error.log", "a") as log:
        log.write(f"[{now}] Script Error: {str(e)}\n")

```

Step 3 — Set Correct Permissions on the Script

Wazuh runs as the wazuh user. The script must be owned by root:wazuh and executable by the wazuh group. Wrong permissions = silent failure.

```
# Run this on: MANAGER (wazuh-soc)
sudo chown root:wazuh /var/ossec/integrations/custom-sms-alert
sudo chmod 750 /var/ossec/integrations/custom-sms-alert

# Verify
sudo ls -la /var/ossec/integrations/custom-sms-alert
# Expected: -rwxr-x-- 1 root wazuh ... custom-sms-alert
```

NOTE: chmod 750 means owner (root) has full access, group (wazuh) can read and execute, others have no access. This is the same permission pattern used for all Wazuh integration scripts.

Step 4 — Add Custom Rules to local_rules.xml

Wazuh's built-in rules 503 (agent started) and 506 (agent stopped) are Level 3. A known Wazuh restriction prevents agent-sourced events from triggering integrations via the filter. The solution: create custom child rules that inherit from 503/506 and assign them a named group tag (sms_notify). The integration then filters by group instead of rule_id.

```
# Run this on: MANAGER (wazuh-soc)
sudo nano /var/ossec/etc/rules/local_rules.xml
```

Add these rules inside the existing block, before :

```
<!-- Agent Stop/Start SMS Rules -->
<!-- These wrap built-in rules 506 (agent stop) and 503 (agent start) -->
<rule id="110600" level="5">
  <if_sid>506</if_sid>
  <description>Agent disconnected - SMS alert</description>
  <group>sms_notify,</group>
</rule>

<rule id="110601" level="5">
  <if_sid>503</if_sid>
  <description>Agent connected - SMS alert</description>
  <group>sms_notify,</group>
</rule>
```

WARNING: Do NOT use 503,506 in the ossec.conf integration block. This will not work due to Wazuh's restriction on agent-sourced events. The group-based approach above is the correct and tested workaround.

Step 5 — Add Integration Blocks to ossec.conf

Two separate integration blocks are required. Wazuh does not support combining and filters with OR logic in a single block.

```
# Run this on: MANAGER (wazuh-soc)
sudo nano /var/ossec/etc/ossec.conf
```

Add both blocks before :

```
<!-- SMS Alert: High Vulnerability (Level 12+) -->
<integration>
  <name>custom-sms-alert</name>
  <hook_url>https://api.rtcom.xyz/onetomany</hook_url>
  <level>12</level>
  <alert_format>json</alert_format>
</integration>

<!-- SMS Alert: Agent ON/OFF via group filter -->
<!-- NOTE: Uses group filter, NOT rule_id – see Step 4 explanation -->
<integration>
  <name>custom-sms-alert</name>
  <hook_url>https://api.rtcom.xyz/onetomany</hook_url>
  <group>sms_notify</group>
  <alert_format>json</alert_format>
</integration>
```

NOTE: The tag must exactly match the filename in /var/ossec/integrations/. If the filename is custom-sms-alert, the name tag must be custom-sms-alert — no extension, exact match.

Step 6 — Validate Config and Restart Manager

```
# Run this on: MANAGER (wazuh-soc)
sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager
# Look for: Active: active (running)

# Verify integration loaded successfully
sudo grep -i "custom-sms" /var/ossec/logs/ossec.log | tail -5
```

Expected output after successful load:

```
wazuh-integratord: INFO: Enabling integration for: 'custom-sms-alert'.
wazuh-integratord: INFO: Enabling integration for: 'custom-sms-alert'.
```

If you see 'File not found inside integrations' instead, go back to Step 2 and confirm the script filename has no .py extension.

Step 7 — Test the Full Flow

Before testing via agent stop/start, confirm the script itself can reach the API. This is the fastest way to isolate whether the problem is in the script or in Wazuh's integration wiring.

Test A — Manual Script Test:

```
# Run this on: MANAGER (wazuh-soc)
# Create a fake alert JSON
cat > /tmp/test_alert.json << 'EOF'
{"rule":{"id":"110600","level":5,"description":"Agent disconnected"},
 "agent":{"name":"rakib-virtual-machine"}}
EOF

# Run the script manually
sudo python3 /var/ossec/integrations/custom-sms-alert /tmp/test_alert.json "" ""

# Check the result
sudo cat /var/ossec/logs/sms_debug.log
```

Successful output in sms_debug.log:

```
[2026-06-20 05:18:12] RuleID:110600 Level:5 Agent:rakib-virtual-machine
Status:200 Response:{"response":{"code":200,"message":"Success"},
"info":{"smsCharge":"0.30","balance":"41.00"}}
```

Test B — Live Agent Test:

```
# Run this on: AGENT (rakib-virtual-machine)
sudo systemctl stop wazuh-agent

# On MANAGER – watch SMS debug log (wait up to 60 seconds)
sudo tail -f /var/ossec/logs/sms_debug.log
```

```
# Start agent again – second SMS should arrive  
sudo systemctl start wazuh-agent
```

NOTE: Wazuh typically detects agent disconnection within 30-60 seconds after the agent stops sending heartbeats.

Troubleshooting

Error: integrations.log is empty after agent stop/start

Cause: Using filter in ossec.conf integration block

Fix: Switch to sms_notify filter and add custom rules 110600/110601 to local_rules.xml as shown in Step 4. This is a known Wazuh restriction.

Error: 'File not found inside integrations' in ossec.log

Cause: Script filename has .py extension

Fix: The file must be named exactly custom-sms-alert with no extension. Run: `cp /var/ossec/integrations/custom-sms-alert.py /var/ossec/integrations/custom-sms-alert` then set permissions again.

Error: Script works manually but not automatically via Wazuh

Cause: Wrong file permissions or ownership

Fix: Run: `sudo chown root:wazuh /var/ossec/integrations/custom-sms-alert` and `sudo chmod 750 /var/ossec/integrations/custom-sms-alert`

Error: Permission denied when reading log file

Cause: Log files are owned by root/wazuh

Fix: Use: `sudo cat /var/ossec/logs/sms_debug.log` instead of plain cat.

Error: Status 200 in log but no SMS received on phone

Cause: API balance is zero

Fix: Log in to rtcom.xyz portal and check your SMS balance. Top up and retry.

Debug Commands:

Check integration daemon loaded correctly

```
sudo grep -i "integrat" /var/ossec/logs/ossec.log | tail -20
```

Check if rules 110600/110601 fire when agent stops

```
sudo grep "110600\|110601" /var/ossec/logs/alerts/alerts.log | tail -10
```

Check SMS API response log

```
sudo cat /var/ossec/logs/sms_debug.log
```

Check Python script errors

```
sudo cat /var/ossec/logs/sms_error.log
```

Check integration execution log

```
sudo tail -20 /var/ossec/logs/integrations.log
```

Quick Reference

Key File Paths

File / Component	Path
Manager main config	/var/ossec/etc/ossec.conf
SMS integration script	/var/ossec/integrations/custom-sms-alert
Custom rules	/var/ossec/etc/rules/local_rules.xml
SMS debug log	/var/ossec/logs/sms_debug.log
SMS error log	/var/ossec/logs/sms_error.log
Integration execution log	/var/ossec/logs/integrations.log
Manager alert log	/var/ossec/logs/alerts/alerts.log
Manager error log	/var/ossec/logs/ossec.log

Useful Commands

What it does	Command
Open integration script	sudo nano /var/ossec/integrations/custom-sms-alert
Open main config	sudo nano /var/ossec/etc/ossec.conf
Open custom rules	sudo nano /var/ossec/etc/rules/local_rules.xml
Restart manager	sudo systemctl restart wazuh-manager
Check manager status	sudo systemctl status wazuh-manager
Watch SMS debug log	sudo tail -f /var/ossec/logs/sms_debug.log
Watch integration log	sudo tail -f /var/ossec/logs/integrations.log
Check integration loaded	sudo grep -i custom-sms /var/ossec/logs/ossec.log tail -5
Test script manually	sudo python3 /var/ossec/integrations/custom-sms-alert /tmp/test_alert.json

Final Setup Checklist

Go through this every time you set this up on a new server:

Task	Done?
Script created at /var/ossec/integrations/custom-sms-alert (no .py extension)	☐
API credentials (acode, api_key, senderid, contacts) filled in the script	☐
Script permissions set: chown root:wazuh + chmod 750	☐
Custom rules 110600 and 110601 added to local_rules.xml with group: sms_notify	☐
Level 12+ integration block added to ossec.conf	☐
Agent ON/OFF integration block with <group>sms_notify</group> added to ossec.conf	☐
Wazuh manager restarted — Active: active (running)	☐
ossec.log shows: Enabling integration for: 'custom-sms-alert' (twice)	☐
Manual script test returned Status: 200 and message: Success	☐
Live agent stop/start test — SMS received on phone	☐

-- End of Document --